Pre-class progress check

By now, you should have

1. Reviewed slides from weeks 1, 2, and 3* (up to 09/12)
2. Checked out code from 1, 2, and 3* (up to 09/12)
3. Attempted Quizzes 1, 2, and 3 (released **Monday 09/15**).

*everything covered so far

1. Checked out slides from week 4 (**today**)

University of Pittsburgh | School of Public Health

Supervised learning (II): Regression
- non-linear regression
- splines and GAMs

BIOST 2155

Class 4
September 19th, 2025

Overview

Two lectures with a 10-minute break

Part 1: 50 minutes

1. **Linearity assumptions:** may fail
2. **Flexible models:** more variance + less bias?
3. **Methods**
    1. Polynomial regression
    2. Step function
    3. Splines

Break: 10 minutes

Part 2: 50 minutes

1. **Methods (continued)**
    1. Local regression
    2. Generalized Additive Models (GAMs)
2. **Comparison of methods**

Venturing beyond linear models

The truth may not be linear — but we sure can approximate it!

- Linear models are relatively simple to describe and implement, and have advantages over other approaches in terms of interpretation and inference.
- The linearity assumption is an *approximation*, and sometimes a **poor one.**
- In week 3, we improved upon OLS using subset selection, regularization, and dimension reduction within the framework of linear models.
- Now, we will relax the linearity assumption but
  - Still don't want to compromise on interpretability
  - Still don't want to do poorly with prediction.

Venturing beyond linear models

Math notation (🤢)

- Augment/replace the inputs $\boldsymbol{X} = (X_1, X_2, \ldots, X_p)^T$ with <u>additional variables</u> (transformations of $\boldsymbol{X}$), and then use linear models with new input features.

$$f(\boldsymbol{X}) = \sum_{m=1}^{M} \beta_m h_m(\boldsymbol{X}), \quad h_m : \mathbb{R}^p \to \mathbb{R}$$

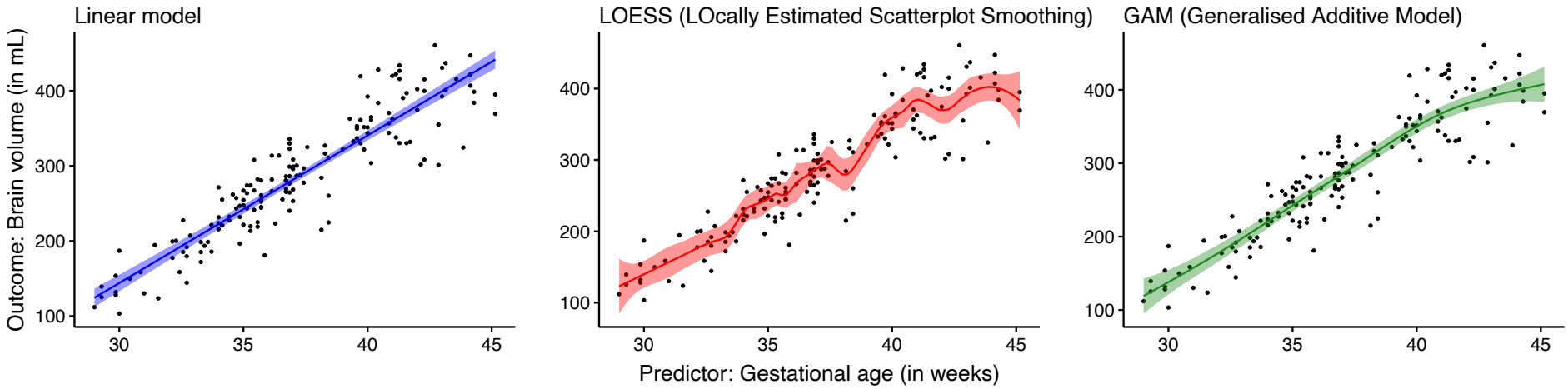
- We will use $f(\boldsymbol{X})$ to approximate $\mathbb{E}(Y \mid \boldsymbol{X})$ (for <u>continuous</u> Y) or $\mathbb{P}(Y = 1 \mid \boldsymbol{X})$ (for <u>categorical</u> Y)

$h_m(\boldsymbol{X}) = X_m$	$h_m(\boldsymbol{X}) = X_j^2$	$h_m(\boldsymbol{X}) = X_i X_j$	$h_m(\boldsymbol{X}) = \log(X_j)$	$h_m(\boldsymbol{X}) = \mathrm{I}(L_m \leq X_j < U_m)$
linear model	Polynomial model	Interactions	Non-linear relationship	Range-specific behavior

fig1

Returning to an old example

Recall: use gestation age to predict brain volume in infants



If we didn't have large values (≥ 40) of gestational age,

a linear model would be **pretty good**!

Polynomial regression

Pulling more information from predictor(s)

- Create new variables $X_1 = X$, $X_2 = X^2$,... and then treat as multiple linear regression (need to specify d):

$$y_i = \beta_0 + \beta_1 x_i + \beta_2 x_i^2 + \beta_3 x_i^3 + \cdots + \beta_d x_i^d + \epsilon_i$$

Usually $d \leq 4$

- Not interested in (*higher*) order coefficients; more interested in prediction at x_0

$$\hat{f}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0 + \hat{\beta}_2 x_0^2 + \hat{\beta}_3 x_0^3 + \hat{\beta}_4 x_0^4$$

- Can get expression and estimate for $\mathbb{V}\left(\hat{f}(x_0)\right)$ and get confidence/prediction interval. **POINT-WISE estimate**.

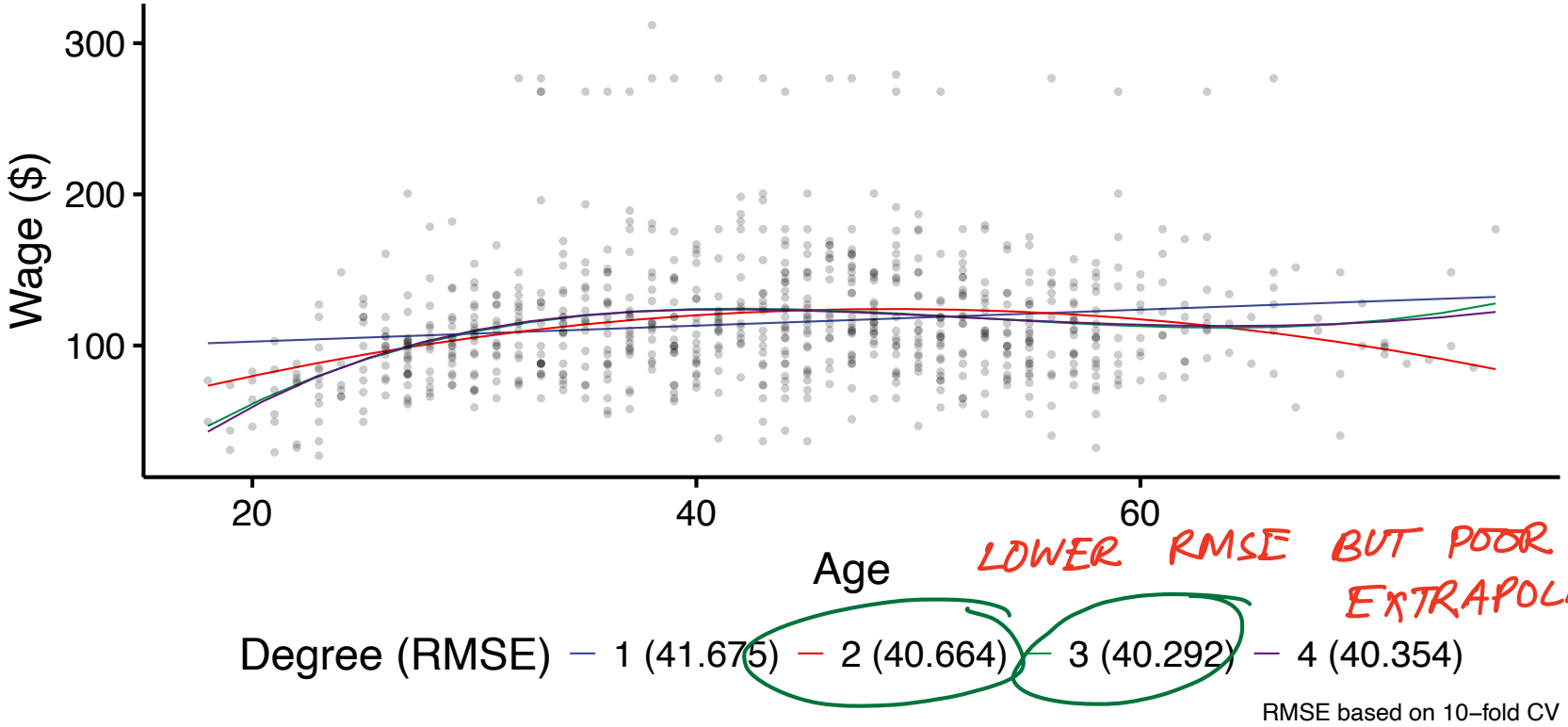
Can use CV for d

Can do separately on several variables— stack the variables into one matrix, and separate out the pieces afterwards

fig2

Polynomial regression

(Marginal) Benefit of using higher-degree polynomials



LOWER RMSE BUT POOR EXTRAPOLATION.

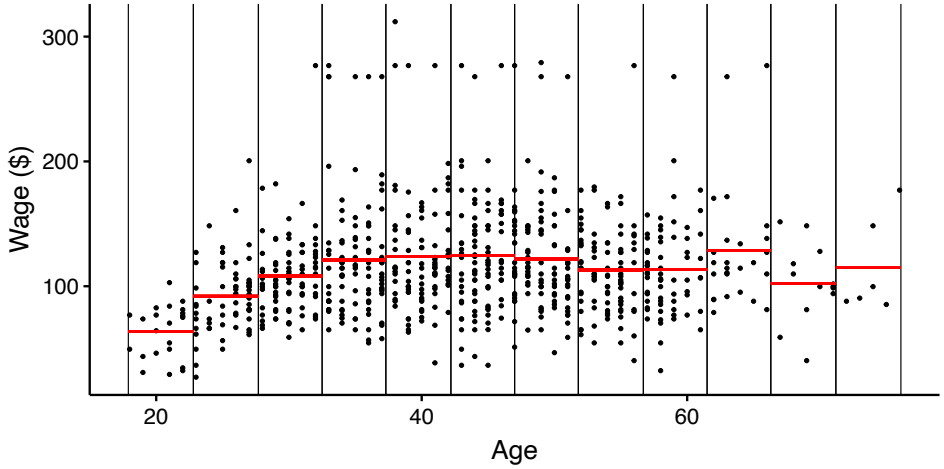
Caveat: polynomials have notorious tail behavior — very bad for extrapolation.

fig3

Step functions

A different kind of parameterization…

- Polynomial functions in a linear model imposes a <u>global structure</u> on the non-linear function of X.
- Use step functions to avoid making a global assumption. *Easy fix: break the range of X into bins, and fit a different constant (local mean) in each bin*.
  - Same as converting a continuous variable into an <u>ordered</u> categorical variable.



Basic understanding: Split range of predictor up into bins and then calculate bin-specific means

Unless there are natural breakpoints in the predictors, piecewise-constant functions can miss important trends.

What if: we fit local polynomials within bins??

Regression splines

Introducing basis functions

- Polynomial/piecewise-constant models: special cases of **basis functions**.
- **Basis functions:** collection of functions that can be applied to predictor X

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \ldots + \beta_k b_k(x_i) + \epsilon_i$$

- The basis functions are known, we fix them BEFORE running our model.
- **Can apply basis functions to many predictors:** stack the (transformed) variables into one matrix, and separate out the pieces afterwards.
- **Polynomial regression:** $b_j(x_i) = x_i^j$ and **piecewise constant:** $b_j(x_i) = \mathrm{I}(c_j \leq x_i < c_{j+1})$.
- Now: piecewise polynomial + splines…

fig4

Piecewise polynomials

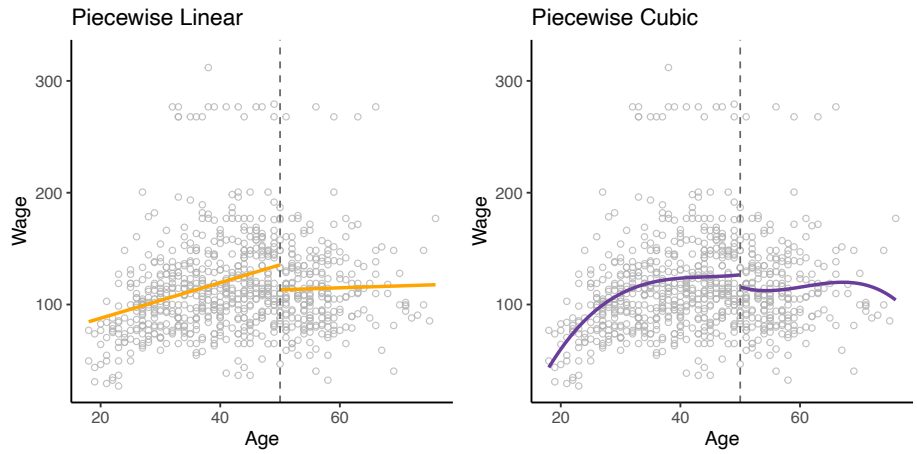
Beyond bin-specific means…

- Instead of a single polynomial in X over whole domain, we use different polynomials within each bin

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \epsilon_i & \text{if } x_i < c; \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \epsilon_i & \text{if } x_i \geq c \end{cases}$$

- May want to ensure continuity (want smooth trend for predictions, harder to explain discontinuity to clinicians)

- Splines (coming up later) have in-built continuity feature.



Discontinuity is bad

Way too many parameters

fig4

Piecewise polynomials

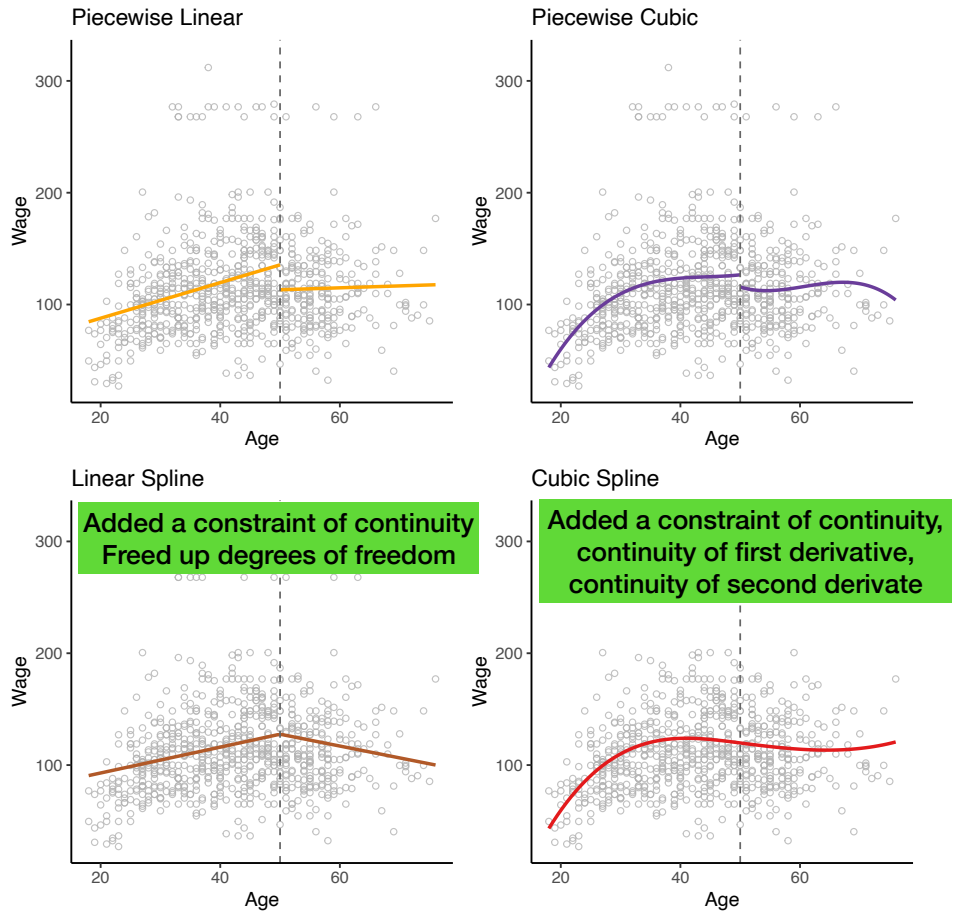
Beyond bin-specific means…

- Instead of a single polynomial in X over whole domain, we use different polynomials within each bin

$$y_i = \begin{cases} \beta_{01} + \beta_{11}x_i + \beta_{21}x_i^2 + \beta_{31}x_i^3 + \epsilon_i & \text{if } x_i < c; \\ \beta_{02} + \beta_{12}x_i + \beta_{22}x_i^2 + \beta_{32}x_i^3 + \epsilon_i & \text{if } x_i \geq c \end{cases}$$

- May want to ensure continuity (want smooth trend for predictions, harder to explain discontinuity to clinicians)

- Splines (coming up later) have in-built continuity feature.



Linear and cubic splines

Piecewise linear + continuous at each knot (cutpoint)

We have to fix (how?) cutpoints $\xi_1, \xi_2, \ldots, \xi_k$

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \ldots + \beta_k b_k(x_i) + \epsilon_i$$

- **Basis functions for <u>linear</u> splines:**

$$b_1(x_i) = x_i$$
$$b_{k+1}(x_i) = (x_i - \xi_k)_+, \quad k = 1, \ldots, K$$

$$(x_i - \xi_k)_+ = \begin{cases} x_i - \xi_k \text{ if } x_i > \xi_k \\ 0 \text{ otherwise} \end{cases}$$

continuity at all cutpoints

$$y_i = \beta_0 + \beta_1 b_1(x_i) + \beta_2 b_2(x_i) + \ldots + \beta_{K+3} b_{K+3}(x_i) + \epsilon_i$$

We have to fix (how?) cutpoints $\xi_1, \xi_2, \ldots, \xi_k$

- **Basis functions for <u>cubic</u> splines:**

$$b_1(x_i) = x_i$$
$$b_2(x_i) = x_i^2$$
$$b_3(x_i) = x_i^3$$
$$b_{k+3}(x_i) = (x_i - \xi_k)_+^3, \quad k = 1, \ldots, K$$

$$(x_i - \xi_k)_+^3 = \begin{cases} (x_i - \xi_k)^3 \text{ if } x_i > \xi_k \\ 0 \text{ otherwise} \end{cases}$$

Continuity + continuity of first two derivatives at all cutpoints

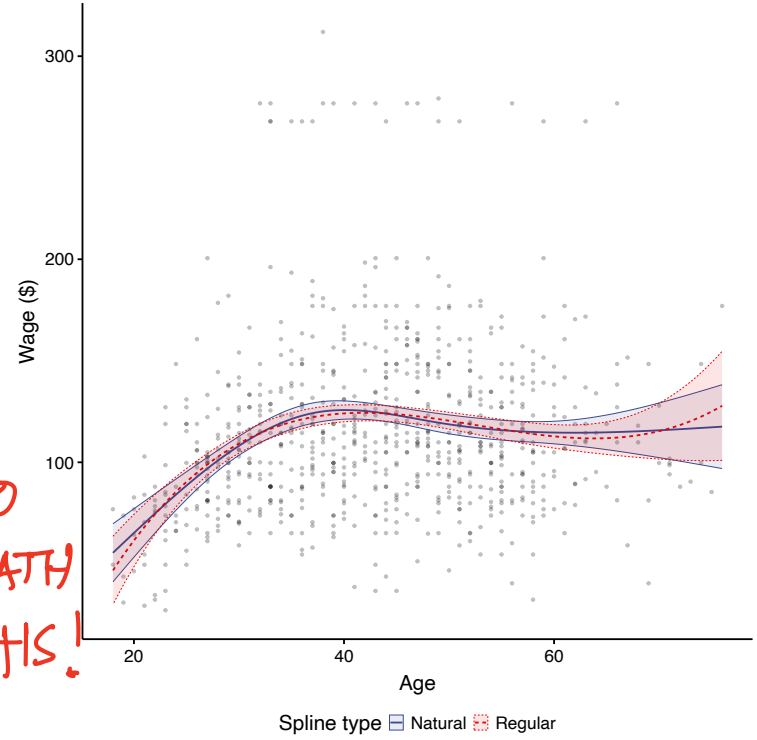
fig5

Natural (cubic) splines

Regular cubic spline + linear extrapolation beyond boundary knots

- A natural spline is a regression spline with additional boundary constraints: the function is required to be linear beyond the boundary
  - Boundary = region where $X <$ smallest knot, or $X >$ largest knot
- Additional constraint (**how many? 2 at either boundary**) means that natural splines generally produce more stable estimates at the boundaries.
- Benefits:
  - Better (more stable) extrapolation
  - Narrower $95\,\%$ CIs.

TRY TO WRITE MATH FOR THIS!



Local regression

Sliding weight function + linear fits + weighted least squares

- Also known as <u>LOESS</u> (Locally Estimated Scatterplot Smoothing).
- Computes a fit at a target point x_0 using <u>local</u> observations only.
    - What is local? Tuning parameter <u>span.</u>
- The process involves:
    1. Find observations "near" the target point x_0, assigning weights to these local points.
    2. Fitting a weighted regression model to the nearby points.
    3. Using the resulting fit to predict the value at x_0
- Different from polynomial regression, which impose a global structure on the entire data set.

Local regression

Algorithm

1. Gather a neighborhood $\mathcal{N}(x_0)$ containing a <u>fraction</u> (aka **span** - tuning needed!) of training points, whose x_i values are closest to x_0 are collected around x_0

2. Assign a weight $K_{i0} = K(x_i, x_0)$ to each $x_i \in \mathcal{N}(x_0)$.

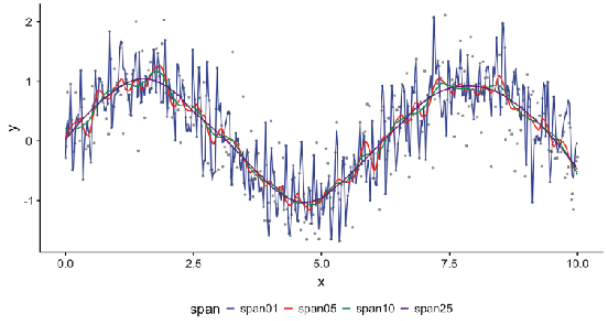
$$K(x_i, x_0) = \sigma^{-1}\phi((x_i - x_0)/\sigma) \text{ or } K(x_i, x_0) = \exp(-\gamma||x_i - x_0||)$$

→ NORMAL PDF

3. Fit a Weighted Least Squares:

$$\left(\hat{\beta}_0, \hat{\beta}_1\right) = \text{argmin} \sum_{i=1}^{n} K_{i0} \cdot (y_i - \beta_0 - \beta_1 x_i)^2$$

4. LOESS-fitted value at x_0 is $\hat{f}_{\text{LOESS}}(x_0) = \hat{\beta}_0 + \hat{\beta}_1 x_0$



Small span: Leads to a very "local" and wiggly fit. The model will have low bias but high variance

Large span: Leads to a smoother, more "global" fit. The model will have higher bias but lower variance.

Local regression

Generalizations and pitfalls

1. Generalizations

    1. Local regression can be extended to multiple predictors ($p > 1$)

    2. It's useful for varying coefficient models, where a model might be global in some variables (age/race) but local in others (like time)

2. Pitfalls

    CAN USE LOESS MARGINALLY FOR EACH PREDICTOR

    1. Local regression performs poorly when $p > 4$

    2. Curse of dimensionality: as p increases, there are very few training observations "close" to any given target point x_0

Generalized additive models

One model to rule them all…

- Replace each linear component with a smooth, non-linear function.

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \ldots + \beta_p x_{ip}$$

$$y_i = \beta_0 + f_1(x_{i1}) + f_2(x_{i2}) + \ldots + f_p(x_{p1})$$

- Strikes a balance between the simplicity of linear regression and the flexibility of fully non-parametric methods.
- Can be used for both regression (quantitative) and classification (qualitative) problems.

lab2

Generalized additive models

Backfitting - algorithm to fit GAMs to data

Do not <u>have</u> to use splines as building blocks for GAMs. Can use local regression, polynomial regression, or any combination

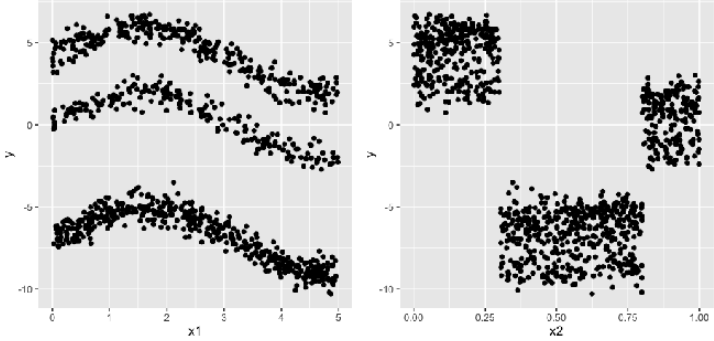
- Instead of fitting all the functions simultaneously, <u>backfitting</u> is an iterative process.
- **Initialization**: Start with initial guesses for each function $f_j(X_j)$
- **Iterative Updates**: Cycle through the predictors from $j = 1,2,\ldots,p$:
    1. Hold all other functions $f_1, \ldots, f_{j-1}, f_{j+1}, \ldots, f_p$ as fixed (or known)
    2. Update the function by fitting a model to the partial residual

$$r_i = y_i - f_1(x_{i1}) - \ldots - f_{j-1}(x_{i(j-1)}) - f_{j+1}(x_{i(j+1)}) - \ldots f_{ip}(x_p)$$

- **Repeat** the cycle until the functions stop changing.

Inference for $\hat{f}_j$?

ANOVA




```
Family: gaussian
Link function: identity

Formula:
y ~ ns(x1, df = 4) + cut(x2, c(-Inf, 0.25, 0.8, Inf))

Parametric coefficients:
                                          Estimate Std. Error t value Pr(>|t|)
(Intercept)                                 4.0248     0.3419  11.772  < 2e-16 ***
ns(x1, df = 4)1                             1.9442     0.3924   4.954 8.54e-07 ***
ns(x1, df = 4)2                            -2.2619     0.4077  -5.548 3.71e-08 ***
ns(x1, df = 4)3                            -0.2730     0.8106  -0.337    0.736
ns(x1, df = 4)4                            -3.8133     0.3768 -10.120  < 2e-16 ***
cut(x2, c(-Inf, 0.25, 0.8, Inf))(0.25,0.8] -9.8332     0.1891 -51.992  < 2e-16 ***
cut(x2, c(-Inf, 0.25, 0.8, Inf))(0.8, Inf] -3.9652     0.2337 -16.967  < 2e-16 ***
---
Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1


R-sq.(adj) =  0.767   Deviance explained = 76.8%
GCV = 6.3021  Scale est. = 6.258     n = 1000
```

Post-class debrief

Today we discussed…

Explored regression techniques that relax the linearity assumption of linear models to capture complex relationships, without sacrificing interpretability. The core idea is to transform or segment predictors and then apply linear modeling methods.

- **Polynomial Regression:** extends linear regression by adding polynomial terms of a predictor as new features. Simple way to model non-linear relationships, but can produce unstable fits at the boundaries of the data.
- **Step Function:** Divides the range of a predictor into distinct bins and fits a different constant (the mean) in each one. Easy to implement but can miss underlying trends unless the breakpoints are naturally occurring in the data.
- **Splines:** Improves on step functions by fitting separate low-degree polynomial functions over different regions of the predictor's range. These functions are connected smoothly at points called knots.
- **Local Regression (LOESS):** A non-parametric method that computes a fit at a target point using only the nearby data points. A weighted regression is performed, giving more importance to closer points. The fit is highly flexible and adapts to local structures
- **Generalized Additive Models (GAMs):** highly flexible and interpretable framework that models the outcome as a sum of arbitrary smooth functions of each predictor. GAMs strike a balance between simplicity and flexibility.

More flexibility in terms of predictive performance can come at the cost of interpretability or inference.

We are ending a module!! Assignment incoming…

Related to this slide…
1. Review ISLR 7
2. Review lab code from ISLR 7

Assignment based on material from first four classes

1. Three sections
    1. Theory (has three bonus problems - **HIGHLY recommended for BHDS**)
    2. Application: Level 1 (basic, will test code proficiency)
    3. Application: Level 2 (advanced, will explore a real data science problem)

Released today after class, due by 11:59PM on Sunday 2025-10-12.

That's all Folks!